

This document specifies the REST endpoints and Kafka event contracts exposed and consumed by each of SplitSpend’s 9 microservices.

1. Auth Service

Handles identity: registration, login, session management, and PIN authentication for high-value transactions.

Endpoints

Method	Endpoint	Description
POST	/api/auth/register	Validate, hash password, create user, emit user.registered
POST	/api/auth/login	Validate credentials, return JWT + refresh token
POST	/api/auth/verify	Verify email OTP
POST	/api/auth/refresh-token	Rotate refresh token
POST	/api/auth/forgot-password	Initiate password reset flow
POST	/api/auth/reset-password	Complete password reset with token
POST	/api/auth/set-pin	Set or update transaction PIN
POST	/api/auth/logout	Revoke refresh token

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	user.registered	New user signed up successfully	User Service	—
Produces	user.verified	OTP email confirmed	Notification Service	—
Produces	user.loggedin	Successful login recorded	Notification Service	—
Consumes	user.created	Sync userId into auth credential record	—	User Service

2. User Service

Stores and manages user profile data. Provides profile lookups to other services.

Endpoints

Method	Endpoint	Description
GET	/api/users/{id}	Retrieve user profile
PUT	/api/users/{id}	Update user profile
DELETE	/api/users/{id}	Soft-delete user
POST	/api/users/{id}/role	Assign role (Admin only)

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	user.created	Profile record created from registration data	Auth Service	—
Produces	user.updated	Profile fields changed	Notification Service	—
Produces	user.deleted	Account soft-deleted	Notification Service	—

Consumes	user.registered	Create profile record after auth registration	—	Auth Service
----------	-----------------	---	---	--------------

3. Wallet Service ★ Source of Financial Truth

The Wallet Service is the ONLY service permitted to move money. Vendor and user-to-user payments are handled as a single atomic operation: debit the payer (budget-first) and credit the recipient — both inside the Wallet, no Paystack involved.

Endpoints

Method	Endpoint	Description
GET	/api/wallets/{userId}	Get MainBalance + BudgetBalance
POST	/api/wallets/credit	Credit a balance (idempotency enforced)
POST	/api/wallets/debit	Debit with budget-first logic; emits correct typed event (idempotency enforced)
POST	/api/wallets/pay	Atomic in-platform payment: debit payer + credit recipient in one ledger operation
POST	/api/wallets/internal-transfer	Internal move: Main ↔ Budget (budget creation / daily expiry)
GET	/api/wallets/{id}/ledger	Paginated ledger with cursor (filterable by type, date)

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	wallet.credited	Balance credited — deposit, vendor receipt, user-to-user receipt, gift, or refund	Transaction Service, Notification Service	—
Produces	wallet.budget.debited	Budget balance deducted (vendor or user-to-user payment, or budget spend)	Budget Service, Transaction Service, Notification Service	—
Produces	wallet.main.debited	Main balance deducted (fallback spend or external transfer pre-debit)	Transaction Service, Notification Service	—
Produces	wallet.budget.transfer.completed	Main→Budget internal move confirmed (budget creation)	Budget Service, Transaction Service, Notification Service	—
Produces	wallet.budget.transfer.failed	Main→Budget internal move failed	Budget Service, Transaction Service, Notification Service	—
Produces	wallet.main.transfer.initiated	Main balance pre-debited for external bank transfer; Transfer Service may proceed	Transfer Service, Transaction Service, Notification Service	—
Produces	wallet.insufficient_funds	All balances exhausted; debit aborted	Transaction Service, Notification Service	—
Consumes	vendor.payment.approved	Execute atomic pay: debit payer (budget-first) + credit recipient Main Balance	—	Vendor Pay Service
Consumes	payment.successful	Credit MainBalance after Paystack deposit confirmed	—	Payment Service
Consumes	budget.created	Transfer Main → Budget	—	Budget Service

Consumes	budget.created	Transfer Main → Budget to fund new budget	—	Budget Service
Consumes	budget.daily.expired	Return unused daily budget amount to Main Balance	—	Budget Service
Consumes	gift.sent	Credit receiver BudgetBalance; debit sender MainBalance	—	Budget Service
Consumes	budget.cancelled	Return cancelled budget remaining to Main Balance	—	Budget Service
Consumes	transfer.failed	Reverse Main Balance pre-debit when external bank transfer failed	—	Transfer Service

4. Budget Service ★ Core Business Logic

Defines spending rules. Does NOT move money directly — it orchestrates via events. Owns daily budget tracking and CRON-based release/expiry. Budget Service only consumes wallet.budget.debited — NOT wallet.main.debited.

Endpoints

Method	Endpoint	Description
POST	/api/budgets/{userId}	Create a new budget plan
GET	/api/budgets/{id}/daily	Get today total spending amount, or remaining
GET	/api/budgets/daily-end-start	Trigger end-of-day expiry and start new day total allocation (CRON-called)
POST	/api/budgets/gift	Send a gift budget to another user
POST	/api/budgets/{id}/cancel	Cancel budget and emit refund; gift budgets cannot be cancelled

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	budget.created	Budget record created; pending wallet fund transfer	Wallet Service	—
Produces	budget.activated	Budget is now live after wallet transfer confirmed	Notification Service	—
Produces	budget.daily.released	Daily funds released (CRON)	Wallet Service, Notification Service	—
Produces	budget.daily.expired	Unused daily funds returned (CRON)	Wallet Service, Notification Service	—
Produces	budget.completed	All funds consumed or end date reached	Notification Service	—
Produces	gift.sent	Gift budget dispatched to receiver	Wallet Service, Notification Service	—
Produces	budget.cancelled	Budget cancelled; funds to be returned to Main	Wallet Service, Notification Service	—
Consumes	wallet.budget.transfer.completed	Activate budget — set Status = Active, emit budget.activated	—	Wallet Service
Consumes	wallet.budget.transfer.failed	Mark budget as Failed; notify user	—	Wallet Service
Consumes	wallet.budget.debited	Update daily spent	—	Wallet Service

		amount via FIFO — only budget debits, never main debits		
--	--	---	--	--

5. Transfer Service ★ NEW

Handles all user-initiated transfers from Main Balance to external Nigerian bank accounts. Coordinates with Wallet Service (to pre-debit the sender) and Paystack (to execute the payout). On failure, triggers a full reversal via Wallet Service.

Endpoints

Method	Endpoint	Description
POST	/api/transfers/initiate	Initiate bank transfer: validate PIN, create record, trigger wallet pre-debit
POST	/api/transfers/webhook	Receive & verify Paystack transfer webhook, emit result event
GET	/api/transfers/{userId}	List all transfer records for a user
GET	/api/transfers/{id}	Get single transfer detail and status
GET	/api/transfers/banks	Fetch supported Nigerian banks list from Paystack
POST	/api/transfers/verify-account	Resolve account name from account number + bank code (Paystack account lookup)

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	transfer.created	Transfer record opened; wallet pre-debit requested	Transaction Service, Notification Service	—
Produces	transfer.completed	Paystack confirmed bank delivery	Transaction Service, Notification Service	—
Produces	transfer.failed	Paystack declined or timed out; triggers wallet reversal	Wallet Service, Transaction Service, Notification Service	—
Consumes	wallet.main.transfer.initiated	Wallet pre-debit confirmed; safe to call Paystack payout API	—	Wallet Service

6. Payment Service — Deposits Only

The sole integration point with Paystack for incoming deposits. When a user bank-transfers money to their SplitSpend virtual account, Paystack fires a webhook here. Payment Service verifies it, records it, and emits payment.successful so Wallet Service can credit the user. It does NOT process vendor payouts, user-to-user payments, or external transfers.

Endpoints

Method	Endpoint	Description
POST	/api/payments/webhook	Receive & verify Paystack deposit webhook; emit payment.successful or payment.failed
GET	/api/payments/verify/{ref}	Manually verify a deposit status against Paystack (idempotent recovery)
GET	/api/payments/{userId}/history	List deposit history for a user

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	payment.successful	Paystack deposit webhook verified and recorded	Wallet Service, Transaction Service, Notification Service	—
Produces	payment.failed	Webhook received but signature verification or processing failed	Transaction Service, Notification Service	—

7. Vendor Pay Service — In-Platform Payments

Manages in-platform payment requests between users: vendor QR/UserID payments, and direct user-to-user sends. In both cases a request is created, the payer approves, and the Wallet Service executes the transfer internally.

Endpoints

Method	Endpoint	Description
POST	/api/pay/request	Create payment request — works for both vendor QR payments and user-to-user sends
POST	/api/pay/{id}/approve	Payer approves request — emits vendor.payment.approved; triggers Wallet atomic pay
POST	/api/pay/{id}/reject	Payer rejects request — no money moves
GET	/api/pay/{userId}/requests	List all payment requests for a user (sent and received)
GET	/api/pay/{id}	Get single payment request detail and current status

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	vendor.payment.requested	Payment request created; payer notified	Wallet Service, Notification Service	—
Produces	vendor.payment.approved	Payer approved — triggers Wallet atomic pay operation	Wallet Service, Transaction Service	—
Produces	vendor.payment.rejected	Payer declined; no money moves	Notification Service	—
Produces	vendor.payment.expired	2-minute window elapsed; no money moves	Notification Service	—
Consumes	transaction.completed	Mark PaymentRequest as Completed	—	Transaction Service
Consumes	transaction.failed	Mark PaymentRequest as Failed	—	Transaction Service

8. Transaction Service

Tracks and coordinates every financial operation end-to-end. Owns the transaction lifecycle state machine (Pending → Processing → Completed/Failed).

Endpoints

Method	Endpoint	Description
POST	/api/transactions	Create transaction record (Pending), emit transaction.created

GET	/api/transactions/{userId}	List all transactions for a user (filterable by type, date, status)
GET	/api/transactions/{id}	Get single transaction detail

Events

Direction	Event	Trigger / Action	Consumed By	Produced By
Produces	transaction.created	Transaction record opened (Pending)	Notification Service	—
Produces	transaction.completed	Full lifecycle success confirmed	Vendor Pay Service, Notification Service	—
Produces	transaction.failed	Any step in the chain failed	Vendor Pay Service, Notification Service	—
Consumes	vendor.payment.approved	Create InPlatformPayment transaction (Pending)	—	Vendor Pay Service
Consumes	wallet.budget.debited	Move transaction to Processing (payer debited from budget)	—	Wallet Service
Consumes	wallet.main.debited	Move transaction to Processing (payer debited from main balance)	—	Wallet Service
Consumes	wallet.credited	Mark transaction Completed — recipient credited or deposit landed	—	Wallet Service
Consumes	wallet.insufficient_funds	Mark transaction Failed — payer had no funds	—	Wallet Service
Consumes	payment.successful	Mark deposit transaction Completed	—	Payment Service
Consumes	payment.failed	Mark deposit transaction Failed	—	Payment Service
Consumes	transfer.created	Create ExternalTransfer transaction record (Pending)	—	Transfer Service
Consumes	transfer.completed	Mark ExternalTransfer transaction Completed	—	Transfer Service
Consumes	transfer.failed	Mark ExternalTransfer transaction Failed	—	Transfer Service

9. Notification Service

Sends SMS, email, and push notifications for all platform events. Supports retry logic and delivery tracking.

Event-to-Message Mapping

Event	Template	Channel
wallet.budget.debited	You paid $\text{\$}$ {amount} from your daily budget ($\text{\$}$ {remaining} remaining today)	Push + SMS
wallet.main.debited	You paid $\text{\$}$ {amount} from your Main Balance ($\text{\$}$ {remaining} remaining)	Push + SMS
wallet.credited	$\text{\$}$ {amount} received — {context: deposit / from {sender}}	Push + Email
wallet.insufficient_funds	Payment of $\text{\$}$ {amount} failed — insufficient balance in both budget and main account	Push + SMS

budget.activated	Your ₦{total} budget is now active. Daily limit: ₦{daily}	Push
budget.daily.released	Your daily budget of ₦{amount} is ready to spend	Push
budget.cancelled	Your budget was cancelled. ₦{amount} returned to Main Balance	Push + Email
vendor.payment.requested	{requester} is requesting ₦{amount}. You have 2 minutes to approve	Push (urgent)
vendor.payment.approved	You approved payment of ₦{amount} to {requester}	Push
vendor.payment.rejected	Payment request of ₦{amount} was declined	Push
vendor.payment.expired	Payment request of ₦{amount} from {requester} expired	Push
gift.sent	{sender} sent you a ₦{amount} budget gift!	Push + Email
transfer.created	Your transfer of ₦{amount} to {recipient} ({bank}) is being processed	Push
transfer.completed	₦{amount} successfully sent to {recipient} at {bank}	Push + SMS
transfer.failed	Transfer of ₦{amount} to {recipient} failed. ₦{amount} has been returned to your Main Balance	Push + SMS

Optional Endpoints

Method	Endpoint	Description
GET	/api/notifications/{userId}	List notifications for user
GET	/api/notifications/{id}	Get single notification detail
GET	/api/notifications/test	Trigger test notification (dev only)